SC2006 Software Engineering

NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-23 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

Chapter 1

Software Development Life Cycle & Process Models

Software engineering exists because building software at scale is not just coding. A single developer can hack together a hundred-line script in an afternoon. A team of fifty building a banking system for millions of users needs discipline, predictability, and quality assurance. The *Software Development Life Cycle* (SDLC) provides that structure, and the choice of process model determines how risk, cost, and time are managed.

1.1 What Is Software Engineering?

Definition 1.1 (Software Engineering)**.** The systematic application of engineering principles to the design, development, maintenance, testing, and evaluation of software and systems.

The term was coined at NATO's 1968 conference during the so-called *software crisis*: projects were late, over budget, and unreliable. The crisis has not disappeared—it has scaled. Modern systems are distributed, concurrent, security-critical, and continuously evolving. Engineering discipline is what separates a prototype from a product that can be operated safely for a decade.

Key characteristics of engineered software include:

- **Maintainability** — can be understood and modified by people other than the original authors.
- **Reliability and availability** — fails rarely, degrades gracefully, and recovers without data loss.
- **Efficiency** — uses time, memory, and energy within acceptable bounds.
- **Usability and security** — meets user needs while protecting assets and privacy.
- **Traceability** — every artefact can be traced from requirement to code to test.

Software engineering is not just process; it is economics. Boehm's classic data shows that fixing a requirements defect during maintenance costs 100× more than fixing it during requirements. Process exists to catch defects early.

1.2 Phases of the SDLC

Every SDLC model organises the same fundamental phases, differing only in sequencing, iteration, and feedback:

1. **Requirement analysis** — what must the system do and how well?
2. **System and software design** — architecture, components, interfaces, data models.
3. **Implementation (coding)** — translating design into executable code.
4. **Testing and verification** — does the system meet its specification?
5. **Deployment and release** — delivering the system to its operational environment.
6. **Maintenance and evolution** — correcting faults, adapting to new needs, improving qualities.

Although listed linearly, real projects revisit earlier phases constantly. A test failure sends you back to design; a deployment constraint forces a requirement renegotiation. The diagram below captures this iterative reality.

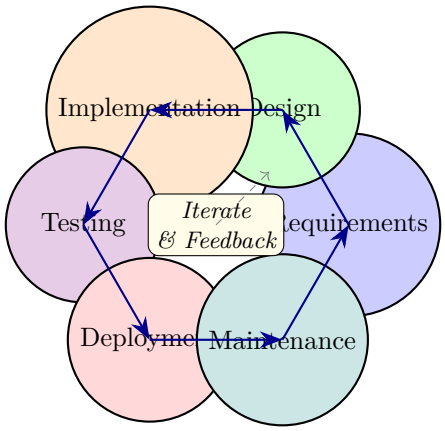

Figure 1.1: SDLC as an iterative cycle – every phase feeds back to predecessors; the centre reminds us that iteration is the norm, not the exception.

1.3 Classical Process Models

1.3.1 Waterfall Model

The waterfall flows strictly top-down: each phase must complete and be signed off before the next begins. It is document-driven and easy to manage—ideal when requirements are stable and well understood, such as safety-critical embedded systems or regulatory reporting platforms.

Strengths: simple to understand, clear milestones, thorough documentation supports audits. Weaknesses: working software appears only at the end; late discovery of requirement errors is enormously expensive; customer feedback is delayed.

The **V-model** is waterfall's disciplined sibling: it pairs each development phase with a test phase (requirements $\leftrightarrow$ acceptance test, high-level design $\leftrightarrow$ system test, detailed design $\leftrightarrow$ integration test, coding $\leftrightarrow$ unit test). This encourages early test planning and traceability.

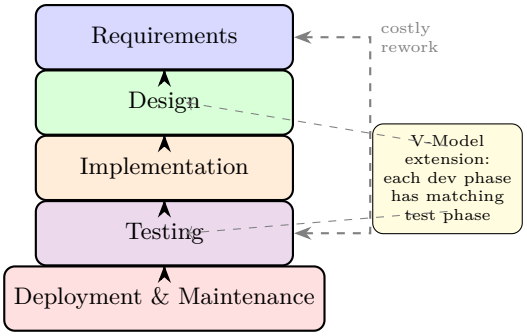

Figure 1.2: Waterfall flow with the costly feedback arc; the V-model mirrors each development phase (requirements, design) with a corresponding test level (acceptance, integration, unit).

1.3.2 Incremental and Iterative Models

Instead of one big delivery, *incremental* development ships the system in slices: increment 1 delivers core features (e.g., login + catalogue), increment 2 adds checkout, increment 3 adds recommendations. Each increment is a complete, usable subset. *Iterative* development refines the whole system through repeated cycles; the first iteration may be a rough prototype that is progressively hardened. In practice most projects combine both: incrementally add scope while iteratively refining quality.

Benefits include early value delivery, early feedback, and reduced risk. The cost is more complex integration and the need for a stable architecture that can accommodate future increments.

1.3.3 Spiral Model

Boehm's spiral (1988) is explicitly *risk-driven*. Each loop around the spiral has four quadrants: (1) determine objectives and constraints, (2) identify and resolve risks (often via prototypes or simulations), (3) develop and test the next increment, (4) plan the next iteration. The radius represents cumulative cost; the angular dimension represents progress.

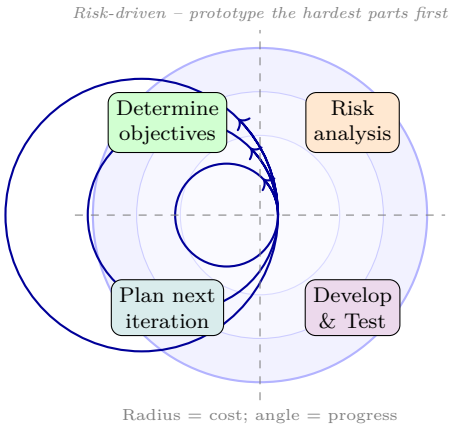

Figure 1.3: Spiral model: each cycle addresses the highest remaining risk with a prototype or analysis before committing to build. Early cycles may be paper prototypes; later cycles are production increments.

Spiral suits large, high-risk, unprecedented systems (aerospace, novel platforms) where risk analysis justifies the overhead. For routine business systems it is often too heavy.

1.3.4 Choosing a Model

Model	Req. stability	Risk handling	Delivery	Overhead
Waterfall	Stable/fixed	Low	Single	Low
V-Model	Stable + safety-critical	Medium	Single	Medium
Incremental	Evolving	Medium	Multiple	Medium
Spiral	Highly uncertain	High (explicit)	Multiple + prototypes	High
Agile (Ch. 8)	Volatile	Continuous	Continuous	Low–Medium

Table 1.1: Guidance for selecting a process model. No single model fits all projects; hybrids are common (e.g., incremental with spiral risk analysis).

A useful heuristic: if you can write a complete SRS on day one, waterfall/V-model may suffice. If you expect requirements to change weekly, choose incremental/iterative or agile. If failure could cost lives or billions, add spiral-style risk analysis regardless of other choices.

1.4 Unified Process and Iterative Refinement

The Unified Process (UP) – and its agile descendant OpenUP – organises development into four phases (Inception, Elaboration, Construction, Transition), each containing multiple iterations that each deliver an executable increment. Unlike waterfall, UP is architecture-centric and risk-driven: the elaboration phase builds an executable architectural prototype that tackles the highest risks before full construction begins.

- **Inception** — scope the project, identify key stakeholders and risks, produce a vision and rough estimate. Go/no-go decision.
- **Elaboration** — stabilise architecture, resolve top risks, produce an executable baseline. Most requirements are now understood.
- **Construction** — incrementally build the remaining features, continuously integrating and testing.
- **Transition** — beta testing, deployment, user training, and handover.

UP makes explicit that disciplines (requirements, design, implementation, testing, project management) run in parallel with varying intensity across phases, rather than as strict sequential gates. Early phases are requirements-heavy; later phases are implementation-heavy. This two-dimensional view (phases $\times$ disciplines) is more realistic than any one-dimensional waterfall chart.

1.5 Process Tailoring and Maturity

No process should be adopted verbatim. Teams tailor by selecting practices that fit context: a five-person startup omits formal change-control boards, while a medical-device team cannot omit traceability audits. Capability Maturity Model Integration (CMMI) provides five maturity levels – Initial, Managed, Defined, Quantitatively Managed, Optimising – to assess and improve process capability. Higher maturity correlates with predictable cost and schedule, but dogmatic adherence without judgement is counter-productive.

1.6 Summary

SDLC models are tools, not religions. Waterfall suits well-understood, regulated domains; spiral suits high-risk R&D; incremental/iterative suits most commercial software. The unifying insight is that process must be chosen deliberately, tailored to context, and continuously improved via retrospectives and metrics. Chapter 8 returns to agile and DevOps as the modern synthesis.

1.7 Case Study: Choosing for a University System

Consider NTU's course-registration system (STARS). Requirements are largely stable (enrolment rules change slowly), but performance under flash-crowd load is critical and failure during add/drop is highly visible. A pure waterfall would delay load testing until the end, risking a catastrophic bottleneck. An incremental approach that delivers registration for one school first, load-tests it with synthetic traffic, and then scales to all schools de-risks the critical quality attribute early. This illustrates that process choice should be driven by the highest risk, not by habit.

Remark 1.1. There is no universally best process. The best process is the one your team understands, tailors honestly, and improves continuously. Ceremony should be proportional to risk.

1.8 Process Metrics and Improvement

What gets measured gets improved. Useful process metrics include velocity (story points per iteration), lead time (idea to production), defect density (bugs per KLOC), and escaped defects (bugs found after release). Retrospectives turn metrics into action: the team inspects data, identifies one or two improvement experiments, and reviews them next cycle. Without measurement, process "improvement" is just opinion.

Remark 1.2. Process tailoring example: a research spike to evaluate a new ML library needs no formal SRS, just a time-boxed experiment and a decision log. Applying full waterfall there would waste weeks.

1.9 Exercises

Exercise 1.1. A hospital management system has fixed regulatory requirements but the customer insists on seeing a working prototype of the patient-record module in month 2. Which SDLC model(s) would you recommend and why? Contrast waterfall and spiral for this scenario.

Exercise 1.2. Explain why the cost of fixing a requirements defect grows exponentially across SDLC phases. Sketch the cost curve (x = phase, y = relative cost) and give a concrete example of a defect caught in testing versus one caught in maintenance.

Exercise 1.3. Distinguish incremental from iterative development. Give an example where a team does one without the other, and an example where both are combined (as in the Unified Process).

Exercise 1.4. The V-model pairs each development phase with a test phase. List the four pairings and explain what each test level validates. When does V-model outperform pure waterfall, and what is its main limitation?

Exercise 1.5. A startup building a social-media app has volatile requirements and needs to ship an MVP in 8 weeks. Argue why a plan-driven model would fail here and outline an alternative process with two increments and specific deliverables per increment.

Chapter 2

Requirements Engineering

If you build the wrong system, no amount of clever design will save it. Requirements engineering (RE) is the disciplined process of discovering, documenting, and validating *what* the system must do and *how well* it must do it. Errors here are the most expensive category of defect—studies attribute over 40% of rework cost to requirements issues.

2.1 Requirements Basics

Definition 2.1 (Requirement)**.** A condition or capability needed by a user to solve a problem or achieve an objective; a condition or set of conditions that must be met by the system to satisfy a contract, standard, or specification.

Two orthogonal dimensions classify every requirement:

- **Functional (FR)** — *what* the system does: features, behaviours, data transformations, business rules. Testable by observing outputs for given inputs.
- **Non-functional (NFR)** — *how well* the system does it: qualities, constraints, compliance, and emergent properties. Often cross-cutting and harder to test.
- **Domain requirements** — constraints from the application domain (e.g., "student GPA is on a 5.0 scale"), which may be functional or non-functional.

Example 2.1. For an online exam system: "The system shall auto-grade MCQ answers and display the score" is functional; "The system shall support 5,000 concurrent examinees with 99.9% uptime and grade within 2 seconds" combines performance, scalability, and availability NFRs.

A common pitfall is stating solutions instead of needs: "The system shall use MySQL" is a design constraint, not a requirement, unless the stakeholder truly mandates it. Good requirements are abstract enough to leave design freedom.

2.2 FURPS+ for Non-Functional Requirements

The FURPS+ taxonomy (Grady, HP) ensures NFR coverage is systematic rather than ad hoc:

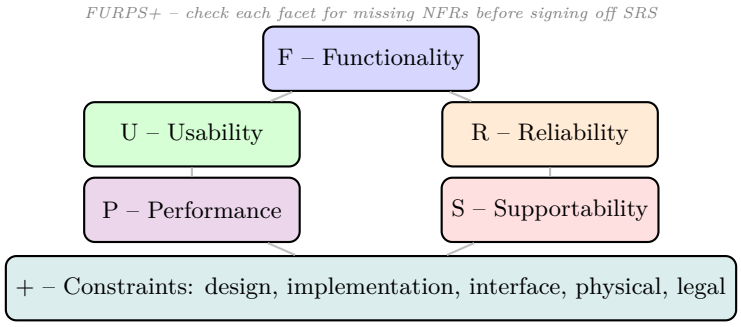

Figure 2.1: FURPS+ model for eliciting non-functional requirements systematically. Each letter prompts a checklist of quality attributes.

Each facet prompts concrete questions. *Usability*: learnability, accessibility (WCAG), localisation. *Reliability*: MTBF, recoverability, fault tolerance. *Performance*: throughput, latency percentiles, capacity. *Supportability*: maintainability, configurability, installability. The "+" captures constraints such as "must run on Android 12+", "must comply with PDPA", or "must integrate with the university SSO".

Well-formed requirements follow **SMART** and **INVEST** (for stories): Specific, Measurable, Achievable, Relevant, Traceable. "System should be fast" fails; "95th-percentile API latency < 300 ms under 1,000 rps sustained for 10 minutes" passes because it is verifiable.

2.3 Requirements Engineering Process

1. **Elicitation** — interviews, workshops, observation, questionnaires, prototyping, brainstorming, and analysis of existing systems. The goal is to surface both stated and tacit needs.
2. **Analysis & negotiation** — resolve conflicts (customer wants X, operator wants not-X), prioritise via MoSCoW (Must/Should/Could/Won't), detect ambiguity and incompleteness.
3. **Specification** — document in an SRS (Software Requirements Specification) with unique IDs, priorities, sources, and verification methods.
4. **Validation** — reviews, walkthroughs, prototypes, and traceability checks to ensure requirements are correct, complete, and consistent.
5. **Management** — version, trace, and control change requests; every change is impact-analysed.

Stakeholders include end users, customers (who pay), developers, testers, operators, and regulators. Missing a stakeholder class is a classic failure mode—operators care about deployability, regulators about compliance, and both are often forgotten until late.

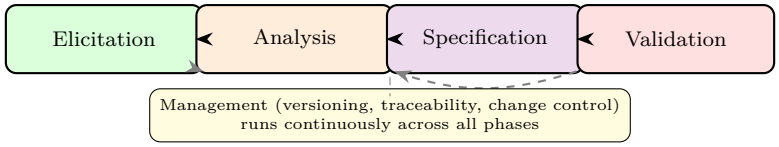

Figure 2.2: RE process is iterative with feedback loops; management activities span the entire lifecycle.

2.4 Requirement Prioritisation and Negotiation

Not all requirements are equal. MoSCoW (Must, Should, Could, Won't) provides a simple prioritisation that forces stakeholders to make trade-offs. Kano analysis adds nuance by distinguishing basic needs (dissatisfiers if absent), performance needs (linear satisfaction), and delighters (non-linear upside). A requirement that is a delighter for one stakeholder may be a basic need for another – negotiation is therefore central to RE.

Techniques for resolving conflicts include win-win negotiation (Boehm), where each stakeholder's win conditions are made explicit and negotiated until every party's must-haves are satisfied, and cost-value prioritisation where each requirement is scored on business value and implementation cost, then plotted to identify high-value, low-cost quick wins versus expensive, low-value items to defer or drop.

2.5 Common Pitfalls

Requirements errors follow predictable patterns: *ambiguity* ("fast" without quantification), *incompleteness* (missing exception flows), *inconsistency* (two requirements contradict), *infeasibility* (violates physics or budget), and *premature design* (stating a solution rather than a need). Inspections using checklists derived from FURPS+ and SMART catch many of these before they become rework.

2.6 Use Cases and User Stories

2.6.1 Use-Case Diagrams

A use case captures a goal-oriented interaction between actors and the system. The diagram shows the system boundary, actors (stick figures), use cases (ellipses), and relationships: `include` (mandatory sub-flow), `extend` (optional/alternative flow), and generalisation.

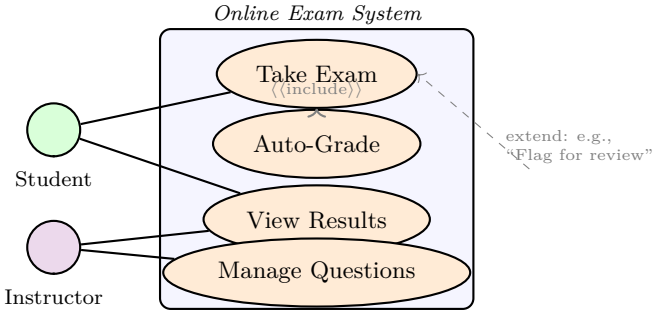

Figure 2.3: Use-case diagram for an online exam system – actors, boundary, and an include relationship (Take Exam always triggers Auto-Grade).

A use-case *specification* details preconditions, main flow (numbered steps), alternative flows, exceptions, and postconditions. For *Take Exam*, the main flow has seven steps from authentication through submission and confirmation; alternative flow 3a handles network timeout with retry, and 5a handles out-of-time auto-submit.

2.6.2 User Stories and Acceptance Criteria

Agile teams often prefer the terse format: *As a* ⟨*role*⟩ *I want* ⟨*goal*⟩ *so that* ⟨*benefit*⟩. Each story carries acceptance criteria in Given/When/Then form and is sized to fit one iteration (INVEST: Independent, Negotiable, Valuable, Estimable, Small, Testable).

Listing 2.1: Acceptance criteria as executable specification (Gherkin-style).

```
# Story: As a student I want auto-grading so that I get instant feedback
# Scenario: MCQ auto-grading
#   Given I have submitted answers for 10 MCQs
#   When the grading service runs
#   Then each correct answer scores 1 point
#   And total score is displayed within 2 seconds
def test_mcq_grading():
    answers = ["A","B","A","C","D","A","B","C","A","D"]
    key     = ["A","B","A","C","D","A","B","C","A","D"]
    assert grade(answers, key) == 10
    assert grade_latency_ms(answers, key) < 2000
```

Stories are not replacements for use cases at scale; they complement them. A common pattern is to start with use cases for scope, then slice each into stories for iterative delivery.

2.7 Requirements Validation Techniques

Validation answers "are we building the right system?" Techniques include formal inspections (Fagan), peer reviews, prototyping (throwaway vs. evolutionary), and model checking of state-based requirements. A lightweight but effective method is the *requirements walkthrough*: the analyst walks stakeholders through each requirement with concrete scenarios, often revealing hidden assumptions. Traceability matrices are validated by ensuring every requirement has at least one test and every test traces to a requirement.

2.8 SRS and Traceability

An SRS (IEEE 830 style) organises requirements hierarchically with unique IDs (REQ-001), priority (MoSCoW), source stakeholder, and verification method (test, inspection, demonstration). A *traceability matrix* links each requirement to design elements, code modules, and tests—so no requirement is lost and no code is unjustified. When a requirement changes, the matrix instantly reveals the blast radius.

Technique	Richness	Cost	Best when
Interview	High	Medium	Stakeholders are accessible, domain is tacit
Workshop	High	High	Conflicts need resolution quickly
Observation	Medium	Low	Users cannot articulate workflow
Questionnaire	Low	Low	Large, distributed population
Prototyping	High	Medium–High	Requirements are uncertain or UI-heavy

Table 2.1: Selecting elicitation techniques – no single technique suffices; combine at least two.

2.9 Elicitation Techniques Compared

Triangulation (using multiple techniques) catches what any single method misses. An interview may reveal that "students want faster feedback," but only observation shows that the bottleneck is manual grade entry, not grading logic – pointing to a workflow requirement rather than an algorithmic one.

Remark 2.1. Non-functional requirements often dominate architecture: "support 10k concurrent users" drives caching, sharding, and load-balancing decisions far more than any single functional requirement.

2.10 Exercises

Exercise 2.1. Classify each as FR or NFR and justify: (a) "System shall encrypt passwords with bcrypt", (b) "System shall allow password reset via email", (c) "Password-reset email shall arrive within 60 s for 99% of requests". Which FURPS+ facet does each NFR belong to?

Exercise 2.2. Write a full use-case specification (preconditions, main flow 5–7 steps, two alternative flows, postconditions) for "Borrow Book" in a library system with Student and Librarian actors. Include an include relationship for "Check Overdue".

Exercise 2.3. Convert the use case in the previous exercise into three user stories with Given/When/Then acceptance criteria. Discuss what is gained and lost in the conversion and when you would keep the use-case form.

Exercise 2.4. A stakeholder says "The system should be user-friendly and handle many users." Rewrite this as two SMART requirements and explain how you would validate each (inspection, test, or demonstration).

Exercise 2.5. Draw a traceability fragment linking REQ-007 "Auto-grade within 2 s" to one design component, one code module, and one test case. Explain how the matrix helps when a change request tightens the limit to 1 s.

Chapter 3

Unified Modeling Language (UML)

UML is the lingua franca for visualising software structure and behaviour. It does not prescribe a process; it gives a standard notation that every stakeholder—from customer to coder—can read. This chapter covers the three diagrams you will use most often in design reviews and technical interviews: class, sequence, and state-machine diagrams.

3.1 UML Overview

UML 2.5 defines 14 diagram types in two families: *structure* diagrams (class, object, component, deployment, package) and *behaviour* diagrams (use case, sequence, activity, state machine, communication). We focus on the core three that appear in almost every design document.

Definition 3.1 (Model vs. Diagram)**.** A *model* is an abstraction of the system capturing selected aspects; a *diagram* is one graphical view of that model. Multiple diagrams can show overlapping elements from different perspectives.

Why model at all? Models enable reasoning before code exists: you can check multiplicities, detect missing associations, and walk through interactions without running anything. A ten-minute whiteboard review of a sequence diagram can prevent a week of integration pain.

3.2 Class Diagrams

A class diagram shows the static structure: classes, attributes, operations, and relationships. It is the primary input to object-oriented code generation.

3.2.1 Notation

- **Class box**: three compartments – name (bold), attributes (with types and visibility), operations (with signatures). Visibility: `+` public, `-` private, `#` protected, `~` package.

- **Relationships**: association (solid line), aggregation (hollow diamond – "has-a" with independent lifecycle), composition (filled diamond – strong ownership, lifecycle bound), inheritance/generalisation (hollow triangle), dependency (dashed arrow), multiplicity (`1`, `0..1`, `*`, `1..*`).
- **Association class**: a class attached to an association link when the link itself carries data.

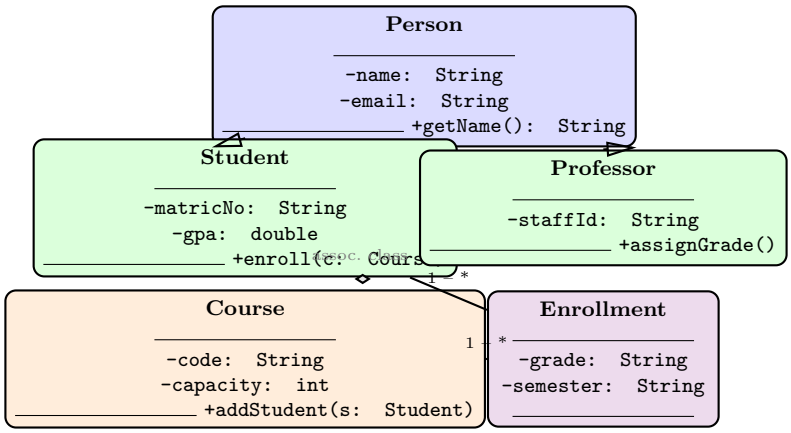

Figure 3.1: Class diagram fragment – inheritance (Person ◁ Student/Professor) and association via Enrollment association class with multiplicities.

Design heuristics: prefer composition over inheritance; keep inheritance hierarchies shallow (depth ≤ 3); use association classes when a link itself carries data (here, Enrollment holds grade and semester that belong to neither Student nor Course alone).

3.2.2 Common Pitfalls

Misusing aggregation/composition changes lifecycle semantics: composition implies the part dies with the whole (deleting a Course deletes its Enrollments), while aggregation does not. Getting this wrong causes either dangling references or unintended cascading deletes. Another pitfall is modelling every database table as a class—classes represent domain concepts, not storage.

3.3 Sequence Diagrams

Sequence diagrams capture *interactions over time*. Time flows downward; each vertical dashed lifeline is an object or component; horizontal arrows are messages: synchronous (filled arrowhead, caller blocks), asynchronous (open arrowhead), and return (dashed).

Good practice: keep diagrams to 5–7 lifelines; use `alt`, `loop`, `opt`, and `ref` fragments sparingly; prefer to split complex flows across multiple diagrams rather than nesting fragments three deep. Each sequence diagram should tell one coherent story.

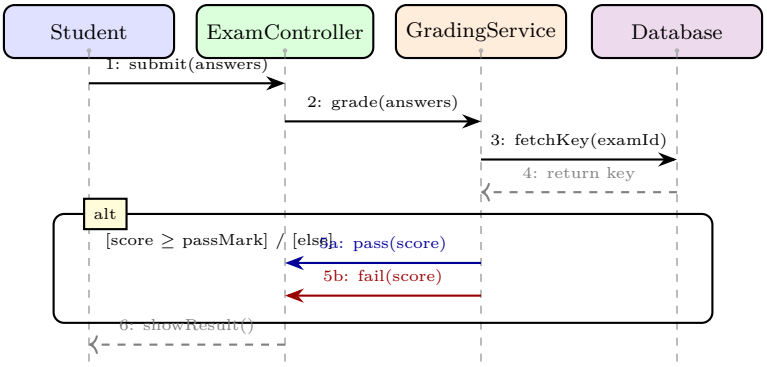

Figure 3.2: Sequence diagram for exam submission – synchronous calls, return messages, and an alt fragment for pass/fail. Time flows downward.

3.4 State-Machine Diagrams

State diagrams model the lifecycle of a *single* object or component. Nodes are states (rounded rectangles); arrows are transitions labelled `event [guard] / action`; the filled circle is the initial state, the bullseye is the final state. Composite states (with nested regions) model hierarchical behaviour.

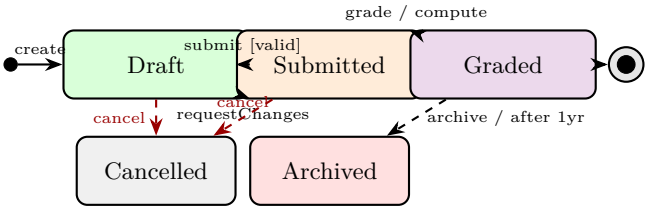

Figure 3.3: State machine for an Exam Attempt – guards in brackets, actions after slash, and terminal states. Cancellation is possible from Draft or Submitted but not after grading.

State machines are invaluable for validating lifecycle invariants: “no grading before submission” becomes a missing transition that the diagram makes obvious. They also generate test cases directly—each path from initial to final state is a scenario to test.

3.5 Activity Diagrams – Workflow View

An activity diagram models control and data flow across actions, decisions, forks/joins (concurrency), and swimlanes (partition by actor or component). Unlike flowcharts, activity diagrams have formal semantics for parallelism: a fork splits one flow into concurrent flows, and the matching join synchronises them. This makes them suitable for modelling business processes and concurrent algorithms.

3.6 Package and Component Diagrams

For large systems, package diagrams show how classes are grouped into packages and how packages depend on each other (dashed arrows for dependencies, solid for imports). Component diagrams go further, showing deployable components with provided/required interfaces (lollipop/socket notation). Both help reason about build order, team ownership, and deployment topology before code exists.

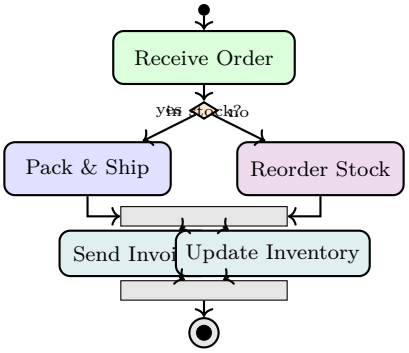

Figure 3.4: Activity diagram with decision, merge, fork/join (concurrent invoice and inventory update), and swimlane-partitionable actions.

3.7 Choosing the Right Diagram

- Need static structure for code generation or database design? Class diagram.
- Need to reason about ordering, concurrency, or API contracts? Sequence diagram.
- Need to validate lifecycle invariants or generate state-based tests? State machine.
- Need high-level workflow across multiple actors? Activity diagram (beyond scope here).

In practice, a design review walks through all three: the class diagram sets the vocabulary, sequence diagrams show how objects collaborate to realise use cases, and state machines ensure each object's lifecycle is sound.

3.8 UML Tooling and Conventions

Modern teams generate skeleton code from class diagrams and reverse-engineer diagrams from code. Round-trip engineering keeps model and code synchronised, but only if the team treats the model as authoritative. Naming conventions matter: class names are singular nouns (`Student`, not `Students`), association names are verbs (`enrollsIn`), and role names clarify multiplicity (`advisor` vs. `advisee`). Consistent layout (e.g., inheritance hierarchies top-down, associations left-to-right) dramatically improves readability during reviews.

Remark 3.1. A diagram that cannot be understood in 60 seconds is too complex. Split it. The goal is communication, not decoration.

Remark 3.2. When hand-drawing UML on a whiteboard, omit attribute types for speed but keep multiplicities and arrowheads precise—they carry the most semantic weight.

3.9 Common Modelling Mistakes

Novices often create "god classes" that know too much, omit multiplicities (leaving cardinality ambiguous), or model every database column as a separate class. A useful review checklist: does every association have multiplicity at both ends? Is every inheritance hierarchy justified by true "is-a"? Are there cycles in package dependencies? Catching these on the whiteboard is minutes; catching them in code is days.

3.10 Exercises

Exercise 3.1. Draw a class diagram for a library with Book, Copy, Member, Loan. Show aggregation vs. composition between Book–Copy and Loan–Member. Add multiplicities and at least one association class. Explain your choice of diamond fill.

Exercise 3.2. Model "withdraw cash from ATM" as a sequence diagram with four lifelines (Customer, ATM, BankServer, Account). Include an alt fragment for insufficient funds and a loop fragment for retrying PIN entry (max 3 attempts). Label synchronous vs. asynchronous messages.

Exercise 3.3. Construct a state-machine diagram for an online Order with states Pending, Paid, Shipped, Delivered, Cancelled, Refunded. Label each transition with event [guard] / action and identify any illegal transition that your diagram prevents.

Exercise 3.4. Explain why sequence diagrams are better than flowcharts for reasoning about distributed-system interactions. Give a concrete example where ordering ambiguity in a flowchart hides a race condition that a sequence diagram would reveal.

Exercise 3.5. A class diagram shows a hollow diamond between Department and Professor. A junior developer implements this as composition (filled diamond, cascading delete). What bug could this introduce when a Department is deleted? How would you catch it in a code review or test?

Chapter 4

Design Principles & Design Patterns

Good design is not about cleverness; it is about managing complexity so that future changes are cheap and safe. This chapter introduces the principles that guide object-oriented design and the classic Gang-of-Four (GoF) patterns that reify those principles into reusable solutions.

4.1 Why Design Matters

Code has two audiences: the machine and the next developer. Design quality shows up in change cost. A well-designed system absorbs a new requirement with a small, localised edit; a poorly designed one requires shotgun surgery across dozens of files. Two metrics predict this:

- **Coupling** — degree of interdependence between modules. Lower is better.
- **Cohesion** — how closely the responsibilities within a module belong together. Higher is better.

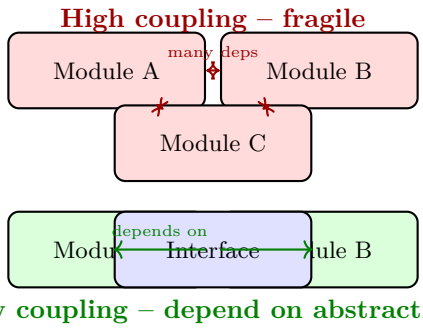

Figure 4.1: Coupling contrast: dense mutual dependencies (top, fragile) versus dependency on a shared abstraction (bottom, stable). Cohesion is high when each module has one focused responsibility.

Common coupling types from worst to best: content > common > external > control > stamp > data > message > no coupling. Cohesion levels from worst to best: coincidental < logical < temporal < procedural < communicational < sequential < functional (goal).

4.2 SOLID Principles

SOLID (Martin, 2000) distils five design rules for maintainable OO systems:

4.2.1 Single Responsibility Principle (SRP)

A class should have one, and only one, reason to change. If a class handles both business logic and persistence, a database migration forces a change to business code. Split them.

4.2.2 Open/Closed Principle (OCP)

Software entities should be *open for extension, closed for modification.* Add new behaviour by adding new code, not by editing existing code. Achieved via abstraction and polymorphism.

4.2.3 Liskov Substitution Principle (LSP)

Subtypes must be substitutable for their base types without breaking correctness. If `Square` extends `Rectangle` but violates the invariant that width and height are independent, LSP is broken.

4.2.4 Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they do not use. Prefer several small, focused interfaces over one fat interface.

4.2.5 Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules; both should depend on abstractions. This is the "D" that enables testability via mocking.

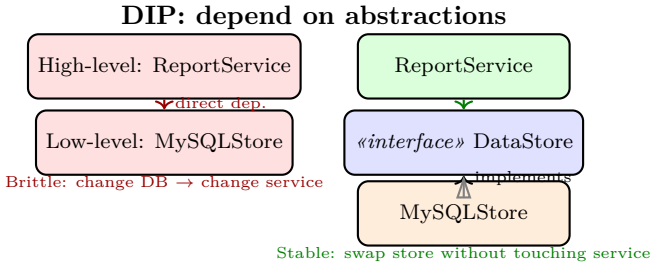

Figure 4.2: Dependency Inversion: before (left, high-level depends on low-level) versus after (right, both depend on an abstraction). The right side enables testing with a fake store.

4.3 GRASP – Assigning Responsibilities

General Responsibility Assignment Software Patterns (GRASP) complement SOLID with heuristics for assigning responsibilities to classes: *Information Expert* (assign to the class that has the data), *Creator* (who should create an object), *Controller* (first object beyond the UI that handles system events), *Low Coupling / High Cohesion*, *Polymorphism*, *Pure Fabrication* (invent a class to achieve low coupling when no domain class fits), *Indirection*, and *Protected Variations*. GRASP is especially useful when drawing sequence diagrams: each message assignment can be justified by a GRASP principle.

4.4 Gang-of-Four Design Patterns

Patterns are proven solutions to recurring design problems. The GoF catalogue has 23 patterns in three families: creational, structural, behavioural. We cover four essentials.

4.4.1 Singleton (Creational)

Ensures a class has exactly one instance and provides global access. Classic use: configuration manager, logger, connection pool.

Listing 4.1: Thread-safe Singleton (double-checked locking).

```
public final class Config {
    private static volatile Config instance;
    private Config() {} // prevent external instantiation
    public static Config getInstance() {
        if (instance == null) {
            synchronized (Config.class) {
                if (instance == null) instance = new Config();
            }
        }
        return instance;
    }
}
```

Caution: singletons hide dependencies and complicate testing. Prefer dependency injection when possible; use singleton only when single-instance semantics are truly required.

4.4.2 Factory Method / Simple Factory (Creational)

Encapsulates object creation so clients depend on an abstraction, not concrete classes. Adding a new product type requires only a new subclass, satisfying OCP.

Listing 4.2: Factory Method – creation delegated to subclasses.

```
abstract class DocumentFactory {
    abstract Document createDocument();
}
```

```
class PdfFactory extends DocumentFactory {
    Document createDocument() { return new PdfDocument(); }
}
class WordFactory extends DocumentFactory {
    Document createDocument() { return new WordDocument(); }
}
// Client code
DocumentFactory f = new PdfFactory();
Document doc = f.createDocument(); // no 'new PdfDocument()' in client
```

4.4.3 Observer (Behavioural)

Defines a one-to-many dependency: when the subject changes state, all observers are notified automatically. Foundation of MVC, event buses, and reactive UI.

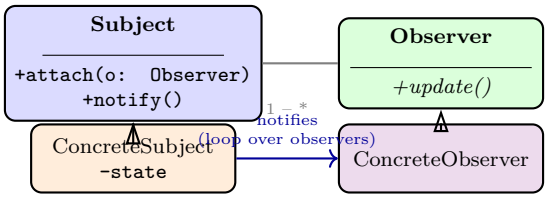

Figure 4.3: Observer pattern structure – subject maintains a list of observers and notifies them on state change. Decouples subject from concrete observer types.

4.4.4 Strategy (Behavioural)

Encapsulates interchangeable algorithms behind a common interface so the algorithm can vary independently of clients. Eliminates conditional chains (`if type==A ... else if type==B`).

Listing 4.3: Strategy – interchangeable payment algorithms.

```
interface PaymentStrategy { void pay(int amount); }
class CreditCardPayment implements PaymentStrategy {
    public void pay(int amount) { /* charge card */ }
}
class PayNowPayment implements PaymentStrategy {
    public void pay(int amount) { /* PayNow transfer */ }
}
class Checkout {
    private PaymentStrategy strategy;
    Checkout(PaymentStrategy s) { this.strategy = s; }
    void complete(int amount) { strategy.pay(amount); }
}
```

Choosing a pattern: use Factory when creation logic is complex or varies; Observer when one state change must propagate to many dependents; Strategy when you have a family of algorithms selected at runtime; Singleton sparingly, when global single-instance semantics are essential.

4.5 Package Design and Metrics

Beyond class-level principles, package-level design governs large systems. Three package principles mirror SOLID at a higher scale: *Reuse–Release Equivalence* (reused packages must be released as units), *Common Closure* (classes that change together belong together), and *Common Reuse* (classes that are not reused together should not be packaged together). Metrics such as efferent/afferent coupling (C_e, C_a) and instability $I = C_e/(C_a + C_e)$ quantify package coupling; distance from the main sequence $D = |A + I - 1|$ (where A is abstractness) flags packages that are either too concrete and unstable or too abstract and stable.

4.6 Exercises

Exercise 4.1. A class `ReportGenerator` formats reports, fetches data from MySQL, and sends emails. Identify the SRP violation, refactor into three classes, and draw the resulting class diagram with dependencies.

Exercise 4.2. Show how Strategy satisfies OCP: start with a `DiscountCalculator` using `if-else` on customer type, then refactor to Strategy. How does adding a new customer type differ before and after?

Exercise 4.3. Implement Observer for a stock ticker where multiple displays (price chart, alert panel, log) update when the price changes. Explain why push vs. pull notification matters for performance when updates are frequent.

Exercise 4.4. A developer makes every service class a Singleton "for convenience." Give two concrete harms this causes for unit testing and concurrency, and propose an alternative using dependency injection.

Exercise 4.5. Compare Factory Method and Abstract Factory. When would you choose one over the other? Illustrate with a UI toolkit that must support both Light and Dark themes across Button and Menu families.

Chapter 5

Implementation & Version Control

Design documents do not run; code does. Implementation translates design into executable software while managing complexity, collaboration, and change. Version control is the safety net that makes collaborative implementation possible at scale.

5.1 From Design to Code

Good implementation respects design intent while adapting to discovered realities. Key practices include:

- **Coding standards** — naming, formatting, and idiom conventions that make code look like it was written by one person. Enforced by linters and formatters.
- **Defensive programming** — validate inputs, fail fast with clear messages, handle edge cases explicitly.
- **Reuse** — prefer well-tested libraries and frameworks over reinventing the wheel; manage dependencies explicitly.
- **Continuous integration** — integrate to mainline frequently (at least daily) to avoid big-bang merge pain.

Listing 5.1: Defensive programming: fail fast with preconditions.

```
public final class Enrollment {
    private final Student student;
    private final Course course;
    public Enrollment(Student s, Course c) {
        if (s == null || c == null) throw new IllegalArgumentException("null arg");
        if (c.isFull()) throw new IllegalStateException("course full: " + c.getCode());
        this.student = s; this.course = c;
    }
}
```

5.2 Version Control Fundamentals

Version control tracks every change, who made it, and why. It enables branching (parallel lines of work), merging (combining branches), tagging (marking releases), and reverting (undoing mistakes). Centralised systems (SVN) have a single server; distributed systems (Git) give every developer a full repository, enabling offline work and flexible workflows.

5.2.1 Git Core Concepts

- **Repository** — the `.git` directory storing all history as a DAG of commits.
- **Commit** — a snapshot identified by a SHA-1 hash, with a message explaining the change.
- **Branch** — a movable pointer to a commit; cheap and disposable.
- **Remote** — a shared repository (e.g., GitHub) that branches synchronise with via `push`/`fetch`.
- **Staging area (index)** — where you assemble the next commit with `git add`.

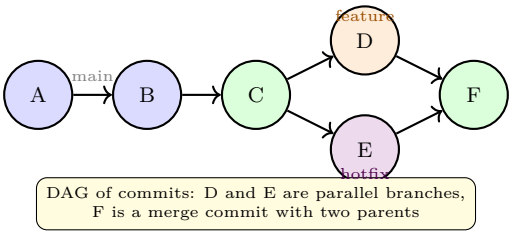

Figure 5.1: Git history as a directed acyclic graph – branches diverge and merge; every commit has a parent (merge commits have two).

5.3 Branching Workflows

5.3.1 Git Flow

Git Flow (Vincent Driessen) structures branches by purpose:

- `main` — production-ready code; every commit is a release.
- `develop` — integration branch for the next release.
- `feature/*` — new features branched from `develop`, merged back when done.
- `release/*` — stabilisation (bug fixes, version bumps) branched from `develop`.
- `hotfix/*` — urgent production fixes branched from `main`, merged to both `main` and `develop`.

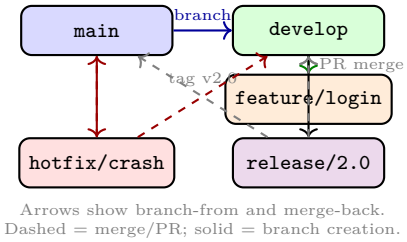

Figure 5.2: Git Flow branching model – disciplined separation of features, releases, and hotfixes. Many teams now prefer the simpler GitHub Flow (feature branches directly to main with CI).

5.3.2 GitHub Flow and Trunk-Based Development

GitHub Flow is lighter: every change is a feature branch off `main`, reviewed via pull request (PR), CI-tested, then merged. Trunk-based development goes further: developers commit small changes directly to `main` (or very short-lived branches <1 day), using feature flags to hide incomplete work. Trunk-based development requires excellent CI and test coverage but yields the fastest feedback.

5.4 Build Systems and Dependency Management

Modern projects automate compilation, testing, and packaging via build tools: `make`, Gradle, Maven, npm, or Bazel. A reproducible build specifies exact dependency versions (lock files), runs in a clean environment (containers), and is triggered by every push via CI. Dependency management distinguishes direct from transitive dependencies; tools like Dependabot flag vulnerable transitive dependencies that would otherwise go unnoticed.

Listing 5.2: Reproducible dependencies with pip-tools.

```
# requirements.in  (direct deps only)
requests==2.31.0
pydantic>=2.0
# Compile to requirements.txt with hashes:
#   pip-compile --generate-hashes requirements.in
```

5.5 Handling Merge Conflicts

Conflicts arise when two branches edit the same lines. Git marks conflicted regions with `<<<<<<<`, `=======`, `>>>>>>>`. Resolution requires understanding *intent*, not just picking one side. Strategies include rebasing feature branches frequently onto `develop` to keep conflicts small, using `git rerere` to reuse past resolutions, and enabling merge queues to serialise concurrent merges safely.

5.6 Code Review and Collaboration

Pull requests are the collaboration hub: code is reviewed, discussed, CI runs automated checks, and only then is it merged. Effective reviews are small (<400 lines), focused, and kind. Automated checks (lint, test, security scan) should run before human review to avoid wasting reviewer time on style nits.

Listing 5.3: Pre-commit checks save reviewer time.

```yaml
# .pre-commit-config.yaml (excerpt)
repos:
  - repo: https://github.com/psf/black
    hooks: [{id: black}]
  - repo: https://github.com/pycqa/flake8
    hooks: [{id: flake8}]
# Run: pre-commit install && pre-commit run --all-files
```

5.7 Reproducibility and Environment Management

"Works on my machine" is a perennial failure mode. Reproducibility requires pinning dependency versions, containerising the runtime (Docker), and documenting setup so a new developer can go from clone to passing tests in one command (`make bootstrap`). Lock files (`package-lock.json`, `requirements.txt` with hashes, `poetry.lock`) ensure every install resolves to identical artefacts; without them, transitive updates can break builds nondeterministically.

Listing 5.4: One-command bootstrap for reproducibility.

```
# Makefile
bootstrap:
  pip install -r requirements.txt
  pre-commit install
  pytest -q
# New developer: git clone <url> && make bootstrap
```

Container-based development goes further: the CI image and the developer image are the same Dockerfile, so environment drift is eliminated. Docker multi-stage builds keep production images small by discarding build-time dependencies.

5.8 Continuous Integration in Practice

A CI pipeline triggers on every push: checkout → install deps → lint → unit test → build → (optional) deploy to staging. Fast feedback is critical—CI should complete in under 10 minutes, or developers stop waiting and context-switch. Techniques to keep CI fast include test parallelisation, caching, and splitting slow end-to-end tests into a nightly job separate from the per-push gate.

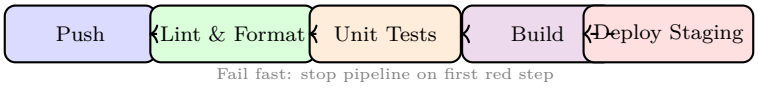

Figure 5.3: Minimal CI pipeline – every push is verified automatically; staging deploy is optional and gated on green tests.

5.9 Semantic Versioning and Releases

Semantic versioning (`MAJOR.MINOR.PATCH`) communicates breaking changes: MAJOR for incompatible API changes, MINOR for backward-compatible features, PATCH for backward-compatible fixes. Tags (`v2.1.0`) mark releases; release notes summarise changes by category. Automated release pipelines cut a tag, build artefacts, and publish to a registry without manual steps, reducing human error.

5.10 Documentation as Code

Architecture Decision Records (ADRs) capture why a decision was made, not just what was decided: context, options considered, decision, and consequences. Stored alongside code (e.g., `docs/adr/001-use-postgres.md`), ADRs prevent future developers from re-litigating settled debates or unknowingly violating constraints.

5.11 Exercises

Exercise 5.1. Explain why "commit early, commit often" with meaningful messages is better than one large commit at the end of a task. What makes a good commit message? Give a bad and a good example.

Exercise 5.2. A hotfix for a production crash must be delivered without including half-finished features on `develop`. Show the Git Flow branch sequence (commands or diagram) that achieves this and explain where the hotfix is merged.

Exercise 5.3. Compare Git Flow, GitHub Flow, and trunk-based development on three criteria: release cadence, team size suitability, and CI requirements. When would you choose each?

Exercise 5.4. A PR contains 1,200 lines across 15 files with no description. List four concrete problems this causes for reviewers and CI, and propose a better decomposition.

Exercise 5.5. Your team uses feature flags for trunk-based development. Explain how a flag lets you merge incomplete code to `main` safely, and describe two risks of long-lived flags that are never removed.

Chapter 6

Software Testing

Testing is the primary defence against defects escaping to users. Yet testing cannot prove the absence of bugs—it can only increase confidence. This chapter organises testing by level, introduces test-driven development, and explains how coverage guides (but does not guarantee) quality.

6.1 Why Test?

Every defect has a cost that grows with time-to-detection. Requirements bugs found in production can cost 100× more than those caught in review. Testing shifts detection earlier. Two complementary perspectives frame the activity:

- **Verification** — are we building the product right? (does code meet spec?)
- **Validation** — are we building the right product? (does spec meet user need?)

Testing also includes *debugging* (finding and fixing the cause after a failure is observed) and *quality assurance* (process measures that prevent defects).

6.2 Levels of Testing

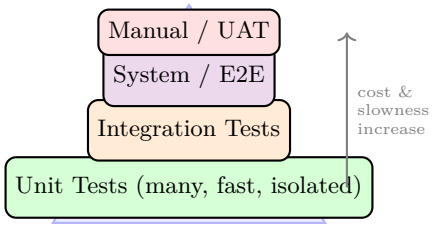

Figure 6.1: Testing pyramid – the foundation is many fast unit tests; integration and system tests are fewer and more expensive. An inverted "ice-cream cone" (many E2E tests) is an anti-pattern.

- **Unit testing** — tests a single class/method in isolation, with dependencies mocked or stubbed. Fast

(milliseconds), precise failure localisation.

- **Integration testing** — verifies interactions between units: does module A call module B correctly? Catches interface mismatches and contract violations. Subsumes component and subsystem integration.
- **System testing** — validates the assembled system against requirements in an environment resembling production. Includes functional and non-functional (performance, security, usability) testing.
- **Acceptance testing (UAT)** — customer-facing validation that the system meets business needs; often manual and scenario-based.

A balanced portfolio respects the pyramid shape: roughly 70% unit, 20% integration, 10% system/E2E by count, with execution time dominated by the lower levels.

6.3 Test Design Techniques

6.3.1 Black-Box Techniques

Design tests from the specification without looking at code:

- **Equivalence partitioning** — divide inputs into classes expected to behave the same; test one representative per class plus boundaries.
- **Boundary-value analysis** — errors cluster at edges; test at `min`, `min+1`, `max-1`, `max`.
- **Decision tables** — enumerate combinations of conditions and expected actions.

6.3.2 White-Box Techniques

Design tests from code structure: *statement coverage* (every line executed), *branch coverage* (every branch taken both ways), *path coverage* (every path exercised, often infeasible). Branch coverage subsumes statement coverage; path coverage subsumes branch.

Listing 6.1: Branch coverage: two tests cover both branches.

```
boolean isEligible(int age, boolean isMember) {
    if (age >= 18 && isMember) return true; // branch 1
    return false;                           // branch 2
}
// Test 1: isEligible(20, true)  -> true  (covers then-branch)
// Test 2: isEligible(16, false) -> false (covers else-branch)
// Branch coverage = 100%, but is it sufficient? Consider (20,false).
```

6.4 Test-Driven Development (TDD)

TDD inverts the usual order: write a failing test first (red), write minimal code to pass (green), then refactor with tests as a safety net. The cycle is deliberately short (minutes).

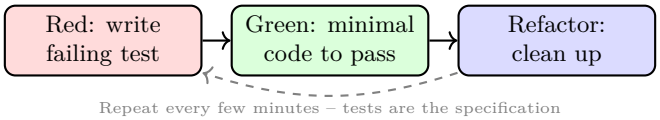

Figure 6.2: TDD red-green-refactor loop. The failing test proves the test can detect absence of the feature; the refactor is safe because tests guard behaviour.

Benefits of TDD: forces testable design (dependency injection, small units), provides regression safety, and yields living documentation. Costs: requires discipline and can be overkill for exploratory spikes. Behaviour-Driven Development (BDD) extends TDD with Given/When/Then language closer to requirements.

6.5 Regression, Flaky Tests, and Test Doubles

A *regression test* guards against reintroducing a previously fixed bug: the bug report becomes a test case that must pass forever. The regression suite grows monotonically; without pruning and parallelisation it eventually becomes the CI bottleneck.

Flaky tests (non-deterministic pass/fail due to timing, randomness, or shared state) erode trust catastrophically: one flaky test trains developers to ignore red builds. Quarantine flaky tests immediately, fix or delete them, and never tolerate "retry until green."

Test doubles isolate the unit under test:

- **Dummy** — passed but never used (fills a parameter).
- **Stub** — returns canned answers.
- **Spy** — records calls for later verification.
- **Mock** — pre-programmed with expectations; verifies interactions.
- **Fake** — lightweight working implementation (e.g., in-memory database).

Prefer fakes and stubs over mocks where possible; over-mocked tests couple to implementation details and break on harmless refactors.

Listing 6.2: Stub vs. mock – prefer stubs for state verification.

```
# Stub: returns canned data, test checks state
class StubPaymentGateway:
    def charge(self, amount): return {"status": "ok", "id": "stub-1"}

def test_checkout_with_stub():
    gw = StubPaymentGateway()
    order = checkout(cart_total=100, gateway=gw)
```

```
    assert order.status == "paid"  # state verification

# Mock: verifies interaction (use sparingly)
from unittest.mock import Mock
def test_checkout_sends_receipt():
    mailer = Mock()
    checkout(cart_total=50, mailer=mailer)
    mailer.send.assert_called_once()  # interaction verification
```

6.6 Measuring Coverage

Listing 6.3: Measuring coverage with pytest-cov.

```
# Run tests with coverage report
#   pytest --cov=mypackage --cov-report=term-missing tests/
# Example output:
#   Name              Stmts   Miss  Cover
#   mypackage/core.py    42      3    93%
#   mypackage/db.py      30      8    73%
def test_add_coverage():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0  # covers negative branch
```

Coverage is a necessary but not sufficient condition for quality. 100% line coverage can still miss logic errors if assertions are weak. Aim for high branch coverage ($> 80\%$) on critical modules, but do not chase 100% at the expense of meaningful assertions. The next section shows how mutation testing measures whether tests actually detect faults.

6.7 Non-Functional and Exploratory Testing

Beyond functional correctness, system testing covers performance (load, stress, soak), security (OWASP ZAP, dependency scanning), usability (heuristic evaluation, A/B tests), and compatibility. Exploratory testing—simultaneous learning, design, and execution by a skilled tester without pre-scripted cases—complements automation by finding bugs that scripted tests miss, especially around usability and edge-case interactions.

6.8 Mutation Testing and Test Quality

Coverage answers "was the line executed?" Mutation testing answers "would the test catch a bug there?" A tool (e.g., mutmut, PIT) injects small faults (mutants) such as flipping `>` to `>=` or deleting a statement. If tests still pass, the mutant *survives*—indicating a weak test. Mutation score = killed / total mutants. High branch coverage with low mutation score signals tests that execute code but assert weakly.

Listing 6.4: Mutation example – surviving mutant reveals weak assertion.

```
# Original
def is_adult(age): return age >= 18
```

```
# Mutant: age > 18  (off-by-one)
# Test: assert is_adult(20) == True  -- passes for both! Survives.
# Better: assert is_adult(18) == True and is_adult(17) == False  -- kills mutant
```

6.9 Testing in CI – Fast Feedback

Keep the per-push gate under 10 minutes: run unit tests first (fail fast), then integration, then a smoke subset of E2E. Full E2E and performance tests run nightly or on release branches. Parallelise by sharding tests across workers; cache dependencies; use test containers for real databases rather than mocks that diverge from production behaviour.

6.10 Automation and Regression Suite Curation

Automated tests run on every commit via CI, forming a *regression suite* that guards against reintroducing past bugs. As the suite grows, curate it: parallelise by shard, quarantine any newly flaky test immediately (see § Regression above), and promote the slowest end-to-end tests to a nightly job so the per-push gate stays under 10 minutes. Prefer fakes and stubs over mocks where possible, as over-mocked tests become brittle.

6.11 Security and Performance Testing

Security testing includes static analysis (SAST), dependency scanning (SCA), and dynamic scanning (DAST). Performance testing distinguishes load (expected peak), stress (beyond capacity to find breaking point), and soak (sustained load to find leaks). Shift-left: run SAST and dependency checks on every PR so vulnerabilities are caught before merge, not after a breach.

6.12 Acceptance Testing and BDD

Acceptance tests are written from the user's perspective, often in Given/When/Then (Gherkin). Behaviour-Driven Development (BDD) makes acceptance criteria executable: scenarios written with stakeholders become automated tests that fail until the feature is implemented. Tools like Cucumber and Behave bridge natural language and code, keeping requirements and tests synchronised.

Remark 6.1. A test suite that takes an hour will be run once a day; one that takes a minute will be run on every save. Speed is a feature of the test suite, not a luxury.

6.13 Exercises

Exercise 6.1. For a method `grade(score)` where $0 \leq$ `score` ≤ 100 maps to A/B/C/D/F at boundaries 80/60/50/40, design boundary-value tests and state how many tests are needed for full boundary coverage.

Exercise 6.2. A suite has 100% line coverage but a bug still escapes. Give a concrete code example and explain why coverage alone was insufficient. How would mutation testing have caught it?

Exercise 6.3. Walk through one TDD cycle (red–green–refactor) for a method `isLeapYear(year)`. Show the failing test, minimal passing code, and a refactor step. Why must the test fail first?

Exercise 6.4. Distinguish unit, integration, and system tests for an e-commerce checkout flow (cart, payment gateway, inventory). For each level, state what is mocked and what defect it is best at catching.

Exercise 6.5. Your CI pipeline takes 45 minutes, mostly due to E2E tests. Propose a restructuring that keeps per-push feedback under 10 minutes without losing confidence. Use the testing pyramid to justify your split.

Chapter 7

Maintenance, Refactoring & Code Smells

More than 60% of total lifecycle cost is spent after the first release. Maintenance is not an afterthought; it is where most engineering effort and business value reside. This chapter covers maintenance categories, refactoring discipline, and the code smells that signal when refactoring is due.

7.1 Types of Maintenance

- **Corrective** — fix bugs discovered in production (reactive).
- **Adaptive** — adapt to changes in environment: OS upgrades, library deprecations, regulatory changes.
- **Perfective** — improve qualities: performance, usability, or add minor enhancements requested by users.
- **Preventive** — proactively improve structure to prevent future defects (refactoring, adding tests, updating documentation).

Lehman's laws observe that useful systems must continually evolve or become progressively less useful, and that each change increases complexity unless work is done to reduce it. Without preventive maintenance, entropy wins.

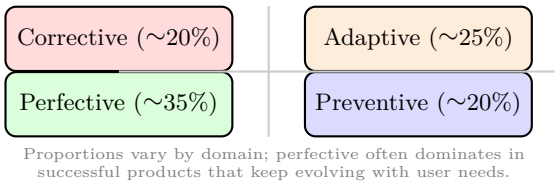

Figure 7.1: Maintenance categories and typical effort distribution. Investing in preventive work reduces future corrective cost.

7.2 Refactoring

Definition 7.1 (Refactoring)**.** A disciplined technique for restructuring existing code, altering its internal structure without changing its external behaviour, to improve readability, reduce complexity, or enable extension.

Refactoring is not rewriting. Each step is small, behaviour-preserving, and guarded by tests. Classic refactorings include *Extract Method*, *Rename*, *Move Method*, *Replace Conditional with Polymorphism*, and *Introduce Parameter Object*. Fowler's catalogue lists over 60.

Listing 7.1: Refactoring: Replace Nested Conditional with Strategy (extract).

```
// Before: long method with branching
double calcPrice(String type, double amt) {
    if (type.equals("REGULAR")) return amt;
    else if (type.equals("VIP")) return amt * 0.8;
    else if (type.equals("STUDENT")) return amt * 0.9;
    else throw new IllegalArgumentException(type);
}
// After: Strategy -- each type is a class, OCP satisfied
interface PricingStrategy { double price(double amt); }
class VipPricing implements PricingStrategy {
    public double price(double amt) { return amt * 0.8; }
}
```

When to refactor: before adding a feature that is hard with the current structure (preparatory refactoring), after shipping to clean up, or continuously as part of TDD's refactor step. Never refactor and add behaviour in the same commit—separate the two so failures are attributable.

7.3 Code Smells

Code smells are surface indicators of deeper design problems. They are not bugs, but they predict bugs.

Smell	Symptom	Refactoring
Long Method	$>$ 30 lines, many locals	Extract Method
Large Class (God Class)	Too many responsibilities	Extract Class
Duplicated Code	Same logic in multiple places	Extract Method / Template
Long Parameter List	$>$ 4 parameters	Introduce Parameter Object
Feature Envy	Method uses another class more than own	Move Method
Data Clumps	Same group of fields recurring	Extract Class
Shotgun Surgery	One change touches many classes	Move Method, Inline
Primitive Obsession	Primitives for domain concepts	Replace with Value Object

Table 7.1: Common code smells and the refactorings that address them.

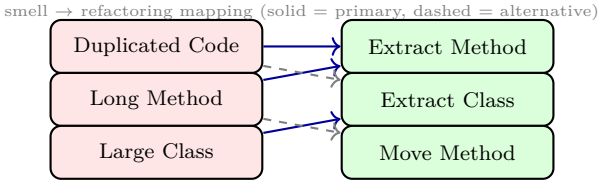

Figure 7.2: Mapping smells to refactorings – each smell has a preferred remediation; the mapping guides code-review feedback.

Detection can be automated: SonarQube, PMD, and Checkstyle flag many smells via metrics (cyclomatic complexity, class size, duplication). But human judgement remains essential—a 40-line method that is perfectly clear is not a smell; a 15-line method with tangled conditionals is.

7.4 Reengineering and Migration

When preventive maintenance is insufficient, *reengineering* systematically transforms a legacy system without changing its functionality: reverse-engineer to recover design, restructure, then forward-engineer. Common strategies include strangler-fig (gradually replace parts behind a facade), branch-by-abstraction (introduce an abstraction layer, migrate incrementally), and big-bang rewrite (high risk, rarely recommended). Migration (e.g., monolith to microservices) follows the same incremental principle: extract one bounded context at a time, with comprehensive contract tests guarding the seam.

7.5 Technical Debt

Technical debt (Cunningham) is the implied cost of future rework caused by choosing an easy, limited solution now instead of a better approach that takes longer. Like financial debt, it accrues interest: every subsequent change is slower. Deliberate, prudent debt (shipping an MVP quickly) can be strategic; inadvertent, reckless debt (copy-pasting without understanding) is corrosive. Manage debt with a visible backlog, allocate 15–20% of each iteration to repayment, and track debt via static-analysis metrics.

Listing 7.2: Automated smell detection in CI.

```
# Example: fail CI if complexity exceeds threshold
# .github/workflows/quality.yml
# - run: flake8 --max-complexity=10 src/
# - run: sonar-scanner -Dsonar.qualitygate.wait=true
# Quality gate blocks merge when debt ratio exceeds 5%
```

7.6 Metrics for Maintainability

Maintainability is measurable. Useful metrics include cyclomatic complexity (McCabe, per method ≤ 10), cognitive complexity (Sonar, penalises nesting more than linear branches), duplication ratio ($< 3\%$), and mean time to change (how long from request to deployed fix). Code churn (lines added/deleted per commit) combined with file age identifies "hot spots": frequently changed, complex files that are the best refactoring targets because improvement there pays off most often.

Remark 7.1. Refactoring without tests is just editing. Ensure a green regression suite before restructuring, and keep each refactoring step small enough to be reviewable in under a minute.

7.7 Legacy Code Strategies

Feathers defines legacy code as code without tests. Strategies for working safely: *sprout* (new behaviour in a new class, called from legacy), *wrap* (extract and test the seam), and *characterisation tests* (write tests that capture current behaviour, even if buggy, before changing it). The goal is to create a safety net incrementally rather than attempting a heroic rewrite.

7.8 Change Impact Analysis

Every maintenance request triggers impact analysis: which requirements, designs, code modules, and tests are affected? Traceability matrices and dependency graphs make this tractable. Techniques include call-graph slicing (what code is reachable from the change), requirements traceability (which REQ-* map to the module), and regression-test selection (run only tests affected by the change for fast feedback, plus full suite nightly). Without analysis, changes cause unintended side effects that surface as production incidents weeks later.

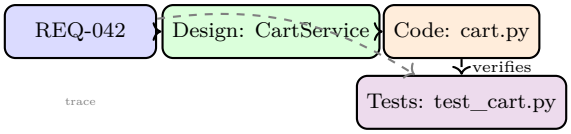

Figure 7.3: Impact analysis via traceability – a requirement change propagates through design, code, and tests; missing any link is a defect.

7.9 Documentation and Knowledge Transfer

Outdated documentation is worse than none: it misleads. Treat docs as code—versioned, reviewed, and tested (e.g., doctests that verify examples still run). Onboarding guides, ADRs, and runbooks (operational playbooks) are the highest-value docs for maintenance. When a senior developer leaves, their undocumented knowledge leaves with them; pair programming and written ADRs mitigate this bus-factor risk.

7.10 Estimating Maintenance Effort

Maintenance effort is estimated differently from greenfield: use change-impact size (lines or points affected), historical velocity for similar changes, and a risk multiplier for poorly understood legacy areas. Planning poker should include a "unknown" card that triggers a time-boxed spike before committing to an estimate. Underestimation is the norm when impact analysis is skipped.

Remark 7.2. Every hour spent on preventive maintenance (tests, refactoring, docs) saves multiple hours of future corrective work. The difficulty is that the saving is invisible until the crisis that did not happen.

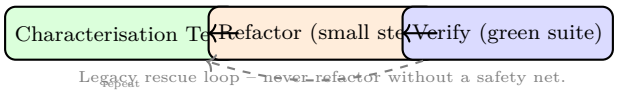

Figure 7.4: Safe refactoring loop for legacy code: capture behaviour first, then restructure incrementally.

7.11 Exercises

Exercise 7.1. Classify each maintenance request: (a) fix crash on empty cart, (b) migrate from Java 11 to 17, (c) add dark mode, (d) extract duplicated payment logic. Which Lehman law does (d) address?

Exercise 7.2. A method has cyclomatic complexity 18 and 60 lines with four levels of nesting. Identify the smells, propose a refactoring sequence (two steps), and explain how you would verify behaviour is preserved.

Exercise 7.3. Distinguish refactoring from rewriting. Why should refactoring commits be kept separate from feature commits? Give a scenario where mixing them hides a bug.

Exercise 7.4. Your team has 30% duplicated code by line count. Propose a concrete, incremental plan to reduce it to $< 10\%$ over four iterations without halting feature work. What metrics would you track?

Exercise 7.5. Explain technical debt with a financial analogy. Give one example of prudent deliberate debt and one of reckless inadvertent debt in a university project context.

Chapter 8

Agile, Scrum, CI/CD & DevOps

The final chapter turns to modern delivery: agile values, Scrum mechanics, and the CI/CD and DevOps practices that enable continuous delivery at scale. These ideas are not opposed to earlier chapters—they complement rigorous requirements, modelling, and testing with faster feedback.

8.1 Agile Manifesto and Principles

Agile (2001) values *individuals and interactions* over processes and tools, *working software* over comprehensive documentation, *customer collaboration* over contract negotiation, and *responding to change* over following a plan. Twelve principles elaborate: satisfy the customer through early continuous delivery, welcome changing requirements, deliver frequently (weeks not months), daily collaboration between business and developers, sustainable pace, and continuous attention to technical excellence.

Agile is not "no process" or "no documentation." It is *just enough* process and documentation, continuously refined.

8.2 Scrum Framework

Scrum is the most widely adopted agile framework, with three roles, three artefacts, and five events.

- **Roles**: Product Owner (owns and prioritises the backlog, maximises value), Scrum Master (facilitates process, removes impediments), Development Team (cross-functional, self-organising, 3–9 members).
- **Artefacts**: Product Backlog (ordered list of all desired work), Sprint Backlog (selected items for the current sprint plus plan), Increment (potentially shippable working software).
- **Events**: Sprint (1–4 weeks, time-boxed), Sprint Planning, Daily Scrum (15 min stand-up), Sprint Review (demo to stakeholders), Sprint Retrospective (process improvement).

Estimation uses story points (relative effort, often Fibonacci: 1, 2, 3, 5, 8, 13) rather than hours, and velocity (points per sprint) for planning. Burndown charts track remaining work within a sprint; burnup charts track

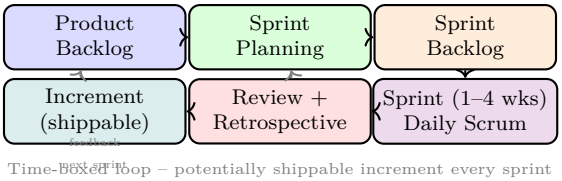

Figure 8.1: Scrum flow – ordered backlog feeds sprint planning; each sprint delivers a shippable increment; review and retrospective close the loop.

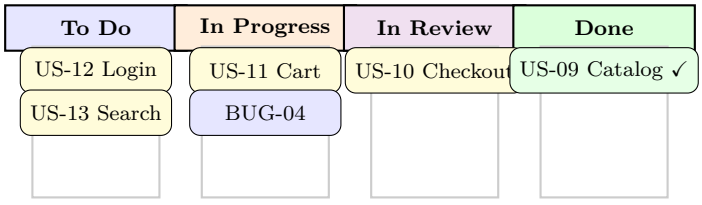

Figure 8.2: Scrum board (task board) – work flows left to right; work-in-progress limits keep flow smooth and make blockages visible.

scope plus progress across releases.

8.3 Kanban and Lean

Kanban visualises workflow on a board with explicit WIP limits per column (e.g., "In Progress ≤ 3"). Unlike Scrum's fixed sprints, Kanban is continuous flow: pull work when capacity allows, optimise for lead time and throughput. Lean software development (Poppendieck) adds principles: eliminate waste, amplify learning, decide as late as possible, deliver as fast as possible, empower the team, build integrity in, and see the whole.

8.4 CI/CD Pipelines

Definition 8.1 (CI/CD)**.** *Continuous Integration*: every push triggers automated build and test. *Continuous Delivery*: every green build is deployable to production at the push of a button. *Continuous Deployment*: every green build is deployed automatically.

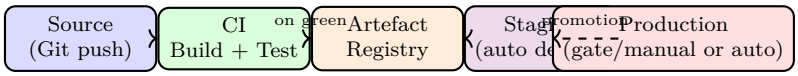

Figure 8.3: CI/CD pipeline – code flows from push through automated stages to production; delivery may require manual approval, deployment does not.

Pipeline-as-code (Jenkinsfile, `.github/workflows/*.yml`) versions the pipeline alongside the product. Deployment strategies include rolling updates, blue-green (two identical environments, switch traffic), and canary (gradual traffic shift with automated rollback on error metrics).

Listing 8.1: GitHub Actions CI/CD (simplified).

```
# .github/workflows/ci.yml
name: CI
```

```yaml
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.11'}
      - run: pip install -r requirements.txt
      - run: ruff check . && pytest --cov=mypackage
      - run: docker build -t myapp:${{ github.sha }} .
```

8.5 DevOps and Collaboration Culture

DevOps unifies Development and Operations: you build it, you run it. Practices include infrastructure as code (Terraform, Ansible), observability (logs, metrics, traces), and blameless post-mortems. The CALMS model summarises: Culture, Automation, Lean, Measurement, Sharing. High-performing DevOps teams (per DORA metrics) deploy many times per day with low change-failure rates and fast recovery, because small, frequent, automated releases are safer than rare, big-bang releases.

8.6 Scaling Agile and Hybrid Models

Single-team Scrum does not automatically scale to 50 developers. Scaling frameworks include SAFe (Agile Release Trains, PI planning), LeSS (minimal extension of Scrum to multiple teams sharing one backlog), and Nexus (Scrum-of-Scrums with an integration team). All emphasise that scaling is primarily an organisational and architectural challenge, not just a process one: without a modular architecture that enables team independence, more teams create more integration pain regardless of framework.

Hybrid models are common in regulated domains: plan-driven (waterfall-style) at the system level for compliance and safety cases, agile within subsystems for faster iteration. The key is to be explicit about which practices are plan-driven and which are agile, and why.

Listing 8.2: Feature flag for trunk-based development.

```python
# flags.py
FLAGS = {"new_checkout": False}  # toggle without redeploy

def checkout(cart):
    if FLAGS["new_checkout"]:
        return new_checkout_flow(cart)  # incomplete work hidden
    return legacy_checkout(cart)

# Enable gradually: 1% -> 10% -> 100% with monitoring
```

8.7 Metrics that Matter – DORA and Flow

DORA identifies four key metrics for delivery performance: deployment frequency, lead time for changes, change failure rate, and mean time to recovery (MTTR). Elite performers deploy on demand, with lead times under an hour and recovery under an hour, because small batch sizes and automation reduce both failure rate and blast radius. Flow metrics (throughput, WIP, cycle time) complement DORA by making bottlenecks visible on the board.

Remark 8.1. Adopting Scrum without engineering practices (CI, TDD, refactoring) yields "agile in name only": sprints produce demo-ware that cannot be shipped. Technical excellence and process agility must advance together.

8.8 Retrospectives and Continuous Improvement

The retrospective is the engine of improvement: what went well, what did not, one experiment to try next sprint. Effective retrospectives are blameless, data-informed (use metrics, not anecdotes), and produce at most two action items with clear owners. Without this feedback loop, process stagnates regardless of framework.

8.9 Putting It Together

A mature team might run Scrum sprints for planning, trunk-based development with feature flags for coding, CI on every push, continuous delivery to staging, and canary deployment to production with automated rollback—all while modelling critical flows in UML and guarding quality with a testing pyramid. Process is not one choice but a coherent combination.

8.10 Contract and Requirements in Agile

Agile does not eliminate requirements; it reframes them. Fixed-price, fixed-scope contracts conflict with "welcome changing requirements." Agile contracts fix time and cost but allow scope to vary within a prioritised backlog, or use incremental funding (fund one increment at a time, continue if value is demonstrated). Requirements become a continuously refined backlog rather than a frozen SRS, but traceability and acceptance criteria remain essential.

8.11 Choosing Between Agile and Plan-Driven

No single approach fits all contexts. Use plan-driven (waterfall/V-model) when requirements are stable, compliance demands traceability, and failure cost is high. Use agile when requirements are volatile, fast feedback is valuable, and the architecture allows incremental delivery. Many organisations run a hybrid: plan-driven at the portfolio level, agile within teams. The decision should be explicit and revisited as context evolves.

Remark 8.2. The most important agile practice is the retrospective. Without it, teams repeat the same mistakes every sprint and agile becomes mere ceremony.

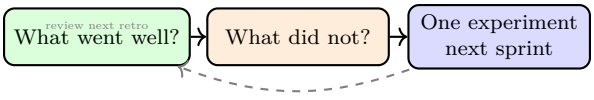

Figure 8.4: Retrospective structure – focus on one actionable experiment, not a laundry list that is never followed up.

Remark 8.3. DevOps is a culture before it is a toolchain. Buying a CI server without changing how dev and ops collaborate yields automation without improvement.

8.12 Exercises

Exercise 8.1. Contrast Scrum roles (Product Owner, Scrum Master, Development Team) with traditional project roles (Project Manager, Analyst, Tester). Who is accountable for maximising product value, and why is the Scrum Master not a manager?

Exercise 8.2. A team completes 28, 32, 30 story points in three sprints. They have 60 points remaining and a 2-sprint deadline. Using velocity, assess feasibility and propose two concrete scope or process adjustments.

Exercise 8.3. Explain the difference between continuous delivery and continuous deployment. When would you choose delivery (manual gate) over deployment (fully automatic)? Give a domain example for each.

Exercise 8.4. Design a CI/CD pipeline for a Python microservice: list stages, triggers, quality gates, and deployment strategy (blue-green vs. canary). Where would you place security scanning and load testing?

Exercise 8.5. A team has no WIP limits and 15 items "In Progress" for a 5-person team. Diagnose the problem with Little's Law intuition (lead time $\propto$ WIP/throughput) and propose a Kanban intervention with specific WIP limits and expected effect.